
Examen
Algorithmique et programmation en python
Durée : 1H45

Exercice 1

12pts

On considère la section de code suivante :

```
1  ###
2  #Random database for student academic performance analysis
3  from random import randint, sample
4  std_nbr=10 #nombre d'étudiants
5  DS_nbr=3 #nombre de devoir surveillé
6  TP_nbr=2 #nombre de TP
7  Att_nbr=3 #nombre de séances
8  days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']
9  min_att = {'Present': range(0,5), 'Late' : range(5,11), 'Very Late': range(11,21), 'Absent': range(21,120)}
10 dic_att = {'Present': 1.0, 'Late': 0.95, 'Very Late': 0.9, 'Absent': 0.87}
11 database = {'Students' : ['Student_'+str(i) for i in range(1,std_nbr+1)],
12             'Genders' : [sample(['M', 'F'],1)[0] for step in range(std_nbr)],
13             'Groups' : [sample(['A', 'B', 'C', 'D'],1)[0] for x in range(std_nbr)],
14             'DS'      : [[randint(6,20) for i in range(std_nbr)] for step in range(DS_nbr)],
15             'TP'      : [
16                 [sample([randint(6,12),randint(10,16),randint(14,20)],1)[0] for i in range(std_nbr)]
17                 for step in range(TP_nbr) ],
18             'Attendance' : {
19                 sample(days, 1)[0] :
20                 [sample([randint(0,25),randint(0,25),0,5],1)[0] for i in range(std_nbr)] for i in range(Att_nbr)
21             }
22 }
```

L'exécution de cette section crée une base de données in-memory nommée « database ». Cette base constitue les données aléatoires structurées sous la forme d'un dictionnaire d'un ensemble d'étudiants connus par leurs noms (key='Students') , leurs sexes (key='Genders'), leurs groupes (key='Groups'), leurs notes pour 3 DS (key='DS'), leurs notes pour 2 TP (key='TP') et leurs retards/présences à 3 séances de cours (key='Attendance').

En s'appuyant sur les structures de données natives de Python notamment les **listes** et les **dictionnaires** et en utilisant le plus possible les **fonctions lambda** et les **fonctions d'ordre supérieur** notamment les fonctions **reduce**, **filter** et **map**, répondre aux questions suivantes en fournissant des solutions concises et efficaces :

1. Pour chaque groupe, extraire la liste des étudiants. Le résultat doit être un dictionnaire de la forme :

```
{
    'A': [list_student_Group_A],
    'B': [list_student_Group_B], ...
}
```
2. Pour chaque étudiant, calculer la note finale en utilisant la formule : $\text{Note_finale} = (\text{moy_ds} * 0.6 + \text{max_tp} * 0.4) * \text{taux_retards}$. Le résultat produit doit être sous la forme :

```
{'Student_1': Note_finale1, 'Student_2': Note_finale2, ...}
```

- Pour chaque séance, sélectionnez le nom des étudiants absents pour chaque groupe. Le résultat doit être un dictionnaire de la forme :


```
{
        'day1': {
          'A': [Student_Abs_A_1, ...],
          'B': [Student_Abs_B_1, ...], ...
        },
        'day2': {...}, ...
      }
```
- Pour chaque groupe, comptez le nombre d'étudiants ayant obtenu plus de (>) 14 dans la note finale.
- Donnez les 3 étudiants avec les notes moyennes les plus élevées de tous les TP.
- Parmi les 3 étudiants, donnez celui ou celle qui a obtenu la note moyenne la plus élevée

Exercice 2

8pts

Une matrice carrée $N \times N$ peut être représentée en python par une liste de liste. Pour indexer l'élément de la ligne i et la colonne j , on écrira : `Nom_matrice[i][j]`, $i=0..N-1$ et $j=0..N-1$.

Ecrire le code Python qui permet de manipuler des matrices carrées de taille variable (matrices dynamiques) par les opérations suivantes :

- la fonction « **get_random_matrice** » qui retourne une matrice carrée aléatoires dont la taille N est reçue en paramètre.
- la fonction « **trace_matrice** » qui reçoit une matrice carrée et qui calcule et retourne sa trace.
- la fonction « **conv_matrices** » qui reçoit deux matrices $A_{N \times N}$ et $B_{(2p+1) \times (2p+1)}$ (où $1 < 2p+1 < N/3$) et qui calcule et retourne le produit de convolution de A par B .

